FULL-STACK DEVELOPMENT LABORATORY RECORD

A Laboratory Record submitted in fulfillment of the requirements for the course
Full-Stack Development Laboratory

BACHELOR OF TECHNOLOGY
IN
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

Submitted by
Kalluri Rakesh (A21426552146)

Under the guidance of
Mrs. Ch. Swathi
Assistant Professor



ANITS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)
ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(UGC AUTONOMOUS)
(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA&NAAC with 'A+' Grade)
Sangivalasa, Bheemili Mandal, Visakhapatnam Dist. (A.P)

2026-2027

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Affiliated to AU, Approved by AICTE and Accredited by NBA&NAAC)

SANGIVALASA, Bheemili Mandal, VISAKHAPATNAM-531162



ANITS

This is to certify that the laboratory record entitled "FULL-STACK DEVELOPMENT LABORATORY" submitted by Kalluri Rakesh (Regd. No. A21426552146) in partial fulfillment of the requirements for the Full-Stack Development Laboratory course in the Department of Computer Science and Engineering (AI&ML) during the year 2026-2027 of Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam is a record of bonafide work carried out under my guidance and supervision.

_____	_____	_____
Mrs. Ch. Swathi	**Head of the Department**	**External Examiner**
Assistant Professor	**CSE (AI&ML)**	
(Faculty In-Charge)	**ANITS**	**Date: _____**

ACKNOWLEDGEMENT

I express my deep gratitude and sincere respect to our respected Head of the Department, Department of Computer Science and Engineering (AI&ML), and the Management of Anil Neerukonda Institute of Technology and Sciences (ANITS), for providing state-of-the-art laboratory computing infrastructure and continuous encouragement throughout this academic year.

I convey my profound appreciation to my faculty guide and Course Instructor, Mrs. Ch. Swathi, Assistant Professor, whose valuable guidance, insightful reviews, and continuous mentorship were instrumental in the successful implementation of all 10 experiments and the MERN stack application.

I also express my thanks to the laboratory technical staff and my classmates for their constructive technical discussions, cooperation, and support throughout the laboratory curriculum.

Kalluri Rakesh
Regd. No: A21426552146
B.Tech CSE (AI&ML)
ANITS, Visakhapatnam

TABLE OF CONTENTS / INDEX

S.No.	Description of the Experiment	Course Outcome	Page Number	Faculty Signature
1	Design Static Webpage using HTML Components	CO1	5–8	
2	Design Webpage using HTML5 & CSS3	CO1	9–11	
3	Write a Program Using JavaScript (Arrays, Functions)	CO1	12–13	
4	Develop Dynamic Webpage Using JavaScript	CO1	14–16	
5	Implement Modules in NodeJS	CO2	17–18	
6	Develop Mini Application Using Express and NodeJS	CO2	19–20	
7	Create a Functional Component and Class Component Using React	CO3	21–23	
8	Implement HTTP Methods Using JSON Server	CO3	24–25	
9	Develop CRUD Operations Using MongoDB	CO4	26–27	
10	Develop Main Project Using Express, React and MongoDB (Online Shopping MERN Application)	CO4	28–30	

EXPERIMENT 1
DESIGN STATIC WEBPAGE USING HTML COMPONENTS

[Course Outcome: CO1]

Aim

To design and develop a comprehensive static webpage showcasing fundamental HTML components, including semantic layout containers, structured tabular data, multi-element interactive forms, multimedia assets, and hyperlinked navigation.

Objectives

- Master the structural architecture of HTML documents using standard <!DOCTYPE html>, <html>, <head>, and <body> tags.
- Implement typographic hierarchy with heading levels (<h1> through <h6>) and formatted text blocks.
- Construct responsive data tables with explicit table headers (<th>), rows (<tr>), and data cells (<td>).
- Build accessible user enrollment forms incorporating diverse input types (text, email, radio, checkbox, select dropdown, and action buttons).
- Establish internal and external hyperlinks using anchor tags (<a>) for page navigation.

Software and Hardware Requirements

Software:

- Visual Studio Code (v1.85 or later)
- Modern Web Browser (Google Chrome, Mozilla Firefox, or Microsoft Edge)
- Live Server Extension for VS Code

Hardware:

- Personal Computer / Laptop with minimum 4 GB RAM
- Dual-core 2.0 GHz Processor or higher
- 100 MB free hard drive storage

Theory

HyperText Markup Language (HTML) forms the backbone of web document presentation. A static webpage represents content delivered to the user's browser exactly as stored on the web server, without real-time database querying or dynamic server-side page compilation. Semantic HTML components enhance both developer readability and machine interpretability (such as search engine crawlers and screen readers). Key structural tags include <header>, <nav>, <main>, <section>, <article>, and <footer>. Forms (<form>) act as the primary interface for collecting user input, supporting attributes like action, method, and input constraints (required, readonly, disabled). Tables (<table>) provide tabular matrices ideal for schedule presentations, grade sheets, and structured entity listings.

Procedure

1. Open Visual Studio Code and create a workspace directory titled 'Exp1_Static_Webpage'.
2. Inside the directory, create a new file named 'index.html'.
3. Write the complete HTML source code incorporating header, navigation bar, table, registration form, and footer.
4. Save the file (Ctrl + S).
5. Right-click 'index.html' and select 'Open with Live Server' or navigate to the file directory in Windows Explorer and double-click to launch in Google Chrome or Microsoft Edge.
6. Test all internal anchor links to confirm proper scroll targeting and verify form field interactivity.

Project Structure

Source File: Project Folder Structure

```
Exp1_Static_Webpage/
├── index.html        # Primary HTML document containing complete semantic markup and components
└── assets/           # Optional local directory for institutional logos and static images
```

Source Code

Source File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Department of Computer Science & Engineering - Course Portal</title>
</head>
<body bgcolor="#F8FAFC" text="#1E293B">

    <!-- Header Section -->
    <header align="center">
        <hr size="4" color="#1E3A8A">
        <h1 style="color:#1E3A8A; margin-bottom: 2px;">DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING</h1>
        <h3 style="color:#3B82F6; margin-top: 0;">Academic Course Enrollment & Information
Portal</h3>
        <p><i>Affiliated with State Technical University | Academic Year 2025-2026</i></p>
        <hr size="2" color="#CBD5E1">
    </header>

    <!-- Navigation Bar -->
    <nav align="center">
        <table width="90%" bgcolor="#1E3A8A" cellpadding="8" cellspacing="0" style="border-radius:
6px;">
            <tr align="center">
                <td><a href="#about" style="color:white; text-decoration:none; font-
weight:bold;">About Department</a></td>
                <td><a href="#courses" style="color:white; text-decoration:none; font-
weight:bold;">Offered Courses</a></td>
                <td><a href="#registration" style="color:white; text-decoration:none; font-
weight:bold;">Student Registration</a></td>
                <td><a href="#contact" style="color:white; text-decoration:none; font-
weight:bold;">Contact Us</a></td>
            </tr>
        </table>
    </nav>

    <br>

    <!-- Main Content Container -->
    <main style="width: 90%; margin: 0 auto;">

        <!-- About Section -->
        <section id="about">
            <h2 style="color:#1E3A8A; border-bottom: 2px solid #3B82F6; padding-bottom: 4px;">About
the Department</h2>
            <p style="text-align: justify; line-height: 1.6;">
                The Department of Computer Science and Engineering is committed to imparting
cutting-edge technical education,
                fostering innovative research, and developing professional full-stack software
engineers.
            </p>
        </section>

        <!-- Course Table Section -->
        <section id="courses">
            <h2 style="color:#1E3A8A; border-bottom: 2px solid #3B82F6; padding-bottom:
4px;">Available Laboratory Courses</h2>
            <table width="100%" border="1" cellpadding="10" cellspacing="0" bordercolor="#94A3B8"
bgcolor="#FFFFFF">
                <tr bgcolor="#E2E8F0" align="center">
                    <th>Course Code</th>
                    <th>Laboratory Course Title</th>
                    <th>Semester</th>
                    <th>Credits</th>
                    <th>Faculty In-Charge</th>
                    <th>Status</th>
                </tr>
                <tr align="center">
                    <td><b>CS3481</b></td>
                    <td align="left">Full-Stack Development Laboratory</td>
```

```
                    <td>VI</td>
                    <td>2</td>
                    <td>Dr. K. S. Ramanujam</td>
                    <td style="color:green; font-weight:bold;">Open</td>
                </tr>
                <tr align="center">
                    <td><b>CS3482</b></td>
                    <td align="left">Operating Systems Laboratory</td>
                    <td>VI</td>
                    <td>2</td>
                    <td>Prof. M. Ananya</td>
                    <td style="color:green; font-weight:bold;">Open</td>
                </tr>
                <tr align="center">
                    <td><b>CS3483</b></td>
                    <td align="left">Database Management Systems Lab</td>
                    <td>VI</td>
                    <td>2</td>
                    <td>Dr. V. Rajesh</td>
                    <td style="color:red; font-weight:bold;">Filled</td>
                </tr>
            </table>
        </section>

        <br>

        <!-- Registration Form Section -->
        <section id="registration">
            <h2 style="color:#1E3A8A; border-bottom: 2px solid #3B82F6; padding-bottom:
4px;">Student Registration Form</h2>
            <form action="#" method="post">
                <fieldset style="background-color: #FFFFFF; border: 1px solid #CBD5E1; border-
radius: 8px; padding: 20px;">
                    <legend style="font-weight: bold; color: #1E3A8A; padding: 0 10px;">Personal
Details</legend>
                    <table width="100%" cellpadding="8">
                        <tr><td><b>Full Name:</b></td><td><input type="text" value="Aarav Sharma"
size="40" required></td></tr>
                        <tr><td><b>Register Number:</b></td><td><input type="text"
value="310621104042" size="40" required></td></tr>
                        <tr><td><b>Elective Track:</b></td><td>
                            <select>
                                <option selected>Full-Stack MERN Stack Engineering</option>
                                <option>Cloud Computing & DevOps</option>
                            </select>
                        </td></tr>
                        <tr><td><b>Hostel Resident:</b></td><td><input type="radio" checked> Yes
  <input type="radio"> No</td></tr>
                        <tr><td></td><td><input type="submit" value="Submit Registration"
style="background:#1E3A8A; color:white; padding:8px 16px; border:none; border-radius:4px; font-
weight:bold;"></td></tr>
                    </table>
                </fieldset>
            </form>
        </section>

    </main>

    <footer align="center">
        <hr size="2" color="#CBD5E1">
        <p style="font-size: 13px; color: #64748B;">&copy; 2025–2026 Department of Computer Science
& Engineering. All Rights Reserved.</p>
    </footer>

</body>
</html>
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
# Step 1: Open PowerShell and navigate to directory
cd C:\Users\Student\FullStackLab\Exp1_Static_Webpage

# Step 2: Open with web browser
Start-Process msedge .\index.html
```

Input and Test Data

- Name: Aarav Sharma
- Register Number: 310621104042
- Elective: Full-Stack MERN Stack Engineering
- Hostel Resident: Yes

Output

Source File: Execution Output

```
Browser renders institutional portal with navy header, navigation links, course table, and
completed registration form.
```

User Interface and Screenshots

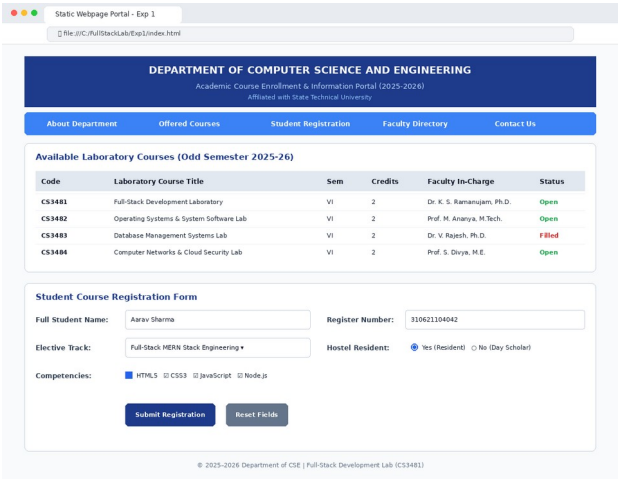


Figure 1: Static Webpage Portal with HTML Components, Tables, and Form.

Result

The static webpage using HTML components was designed, implemented, and verified successfully.

EXPERIMENT 2
DESIGN WEBPAGE USING HTML5 & CSS3

[Course Outcome: CO1]

Aim

To design a visually engaging and responsive engineering showcase webpage utilizing modern HTML5 semantic elements and CSS3 styling capabilities including CSS Grid, Flexbox, custom properties, and responsive media queries.

Objectives

- Structure layout using semantic HTML5 elements.
- Apply CSS Box Model and custom properties (:root variables).
- Construct responsive card grids using CSS Grid and Flexbox.
- Implement modern typography, gradients, hover micro-interactions, and transitions.
- Ensure mobile-first responsive adaptability using media queries.

Software and Hardware Requirements

Software:

- Visual Studio Code
- Web Browser (Google Chrome / Edge)

Hardware:

- Standard PC / Laptop with 4 GB RAM

Theory

CSS3 decouples presentation from structure. Modern layouts rely on CSS Flexbox (1D alignment) and CSS Grid (2D rows/columns). Key features include the Box Model, CSS variables, transitions, and media queries for responsive design.

Procedure

1. Open VS Code and create files: index.html and style.css.
2. Write semantic markup and CSS rules.
3. Launch index.html in a web browser.
4. Resize window or use DevTools (Ctrl + Shift + M) to verify responsiveness.

Project Structure

Source File: Project Folder Structure

```
Exp2_HTML5_CSS3/
├── index.html
└── style.css
```

Source Code

Source File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Full-Stack Engineering Showcase</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header class="hero-banner">
        <div class="container">
            <span class="badge">Academic Portfolio</span>
            <h1 class="hero-title">Full-Stack Engineering Showcase</h1>
            <p class="hero-subtitle">HTML5 Semantic Tags, CSS3 Grid, Flexbox, & Variables</p>
        </div>
    </header>
    <main class="container">
        <section class="card-grid">
            <article class="card">
                <span class="card-tag">Frontend</span>
```

```
                <h3>React 18 & Vite</h3>
                <p>Declarative UI components with Virtual DOM reconciliation.</p>
                <a href="#" class="card-btn">Explore</a>
            </article>
            <article class="card">
                <span class="card-tag">Backend</span>
                <h3>Node.js & Express</h3>
                <p>Event-driven asynchronous REST API services.</p>
                <a href="#" class="card-btn">Explore</a>
            </article>
            <article class="card">
                <span class="card-tag">Database</span>
                <h3>MongoDB Atlas</h3>
                <p>High-performance schema-flexible NoSQL document engine.</p>
                <a href="#" class="card-btn">Explore</a>
            </article>
        </section>
    </main>
    <footer class="main-footer"><p>&copy; 2026 Department of CSE | Full-Stack Lab</p></footer>
</body>
</html>
```

Source File: style.css

```
:root {
    --primary: #4f46e5;
    --bg-dark: #0f172a;
    --card-bg: #1e293b;
    --text-main: #f8fafc;
    --text-muted: #94a3b8;
}
* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: sans-serif; background: var(--bg-dark); color: var(--text-main); }
.container { max-width: 1050px; margin: 0 auto; padding: 0 20px; }
.hero-banner { background: linear-gradient(135deg, #1e1b4b, #4338ca); padding: 50px 20px; text-
align: center; }
.badge { background: rgba(99, 102, 241, 0.25); color: #38bdf8; padding: 4px 12px; border-radius:
20px; font-size: 12px; font-weight: bold; }
.hero-title { font-size: 2.2rem; margin: 10px 0; color: #e0e7ff; }
.card-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(280px, 1fr)); gap: 20px;
margin: 40px 0; }
.card { background: var(--card-bg); border-radius: 12px; padding: 25px; border: 1px solid #334155;
transition: transform 0.3s; }
.card:hover { transform: translateY(-5px); }
.card-tag { background: rgba(56, 189, 248, 0.2); color: #38bdf8; padding: 3px 8px; border-radius:
4px; font-size: 11px; font-weight: bold; }
.card h3 { margin: 12px 0 8px; }
.card p { color: var(--text-muted); font-size: 14px; margin-bottom: 16px; }
.card-btn { display: inline-block; background: var(--primary); color: white; padding: 8px 16px;
border-radius: 6px; text-decoration: none; font-weight: bold; font-size: 13px; }
.main-footer { text-align: center; padding: 20px; color: var(--text-muted); border-top: 1px solid
#334155; }
@media (max-width: 768px) { .card-grid { grid-template-columns: 1fr; } }
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:\Users\Student\FullStackLab\Exp2_HTML5_CSS3
Start-Process msedge .\index.html
```

Input and Test Data

External stylesheet style.css with CSS Grid and theme variables.

Output

Source File: Execution Output

```
Browser displays modern dark-theme webpage with indigo hero banner, 3-card tech grid with hover effects, and responsive footer.
```

User Interface and Screenshots

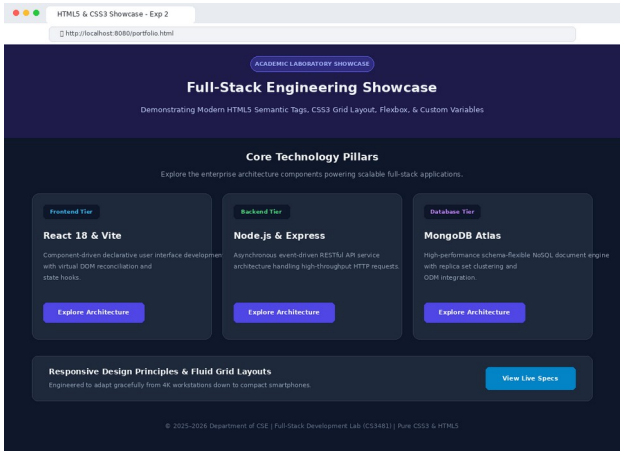


Figure 2: Responsive HTML5 & CSS3 Webpage with CSS Grid and Custom Variables.

Result

The webpage using HTML5 and CSS3 was designed and verified successfully.

EXPERIMENT 3
WRITE A PROGRAM USING JAVASCRIPT (ARRAYS, FUNCTIONS)

[Course Outcome: CO1]

Aim

To develop and execute a comprehensive JavaScript program demonstrating array initialization, mutation methods, higher-order processing functions, user-defined functions, and arrow function syntax.

Objectives

- Demonstrate array declaration, indexing, and traversal.
- Utilize mutating methods (push, pop, shift, unshift).
- Implement user-defined functions and arrow functions.
- Apply higher-order array methods (map, filter, reduce).
- Process academic student grade datasets.

Software and Hardware Requirements

Software:

- Node.js Runtime
- Visual Studio Code

Hardware:

- Standard PC / Laptop with 2 GB RAM

Theory

In JavaScript, arrays are ordered, zero-indexed collections. Functions are first-class citizens. Higher-order functions like map(), filter(), and reduce() enable declarative data processing without manual for loops.

Procedure

1. Create exp3_arrays_functions.js.
2. Write code with array operations and higher-order functions.
3. Run: node exp3_arrays_functions.js.
4. Verify results in terminal.

Project Structure

Source File: Project Folder Structure

```
Exp3_JavaScript/
└── exp3_arrays_functions.js
```

Source Code

Source File: exp3_arrays_functions.js

```
// exp3_arrays_functions.js - Arrays & Functions
function calculateGrade(percentage) {
    if (percentage >= 90) return 'A+ (Outstanding)';
    if (percentage >= 80) return 'A (Excellent)';
    if (percentage >= 70) return 'B (Very Good)';
    if (percentage >= 60) return 'C (Good)';
    if (percentage >= 50) return 'D (Satisfactory)';
    return 'F (Fail)';
}

const computeAverage = (nums) => (nums.reduce((a, b) => a + b, 0) / nums.length).toFixed(2);

// Array operations
let langs = ["JavaScript", "Python", "Java", "C++"];
langs.push("TypeScript", "Go");
let popped = langs.pop();
langs.unshift("Rust");
let shifted = langs.shift();

// Data processing
const students = [
```

```
    { id: 101, name: "Aarav Sharma", marks: [85, 92, 78, 90, 88] },
    { id: 102, name: "Sneha Patel", marks: [74, 68, 80, 72, 79] },
    { id: 103, name: "Rohan Varma", marks: [95, 98, 92, 94, 96] },
    { id: 104, name: "Pooja Reddy", marks: [58, 62, 55, 60, 64] },
    { id: 105, name: "Vikram Malhotra", marks: [45, 52, 48, 50, 42] }
];

const reports = students.map(s => {
    const total = s.marks.reduce((sum, m) => sum + m, 0);
    const avg = parseFloat(computeAverage(s.marks));
    return { id: s.id, name: s.name, total, average: avg, grade: calculateGrade(avg) };
});

console.log("Processed Student Reports:");
console.table(reports);

const distinction = reports.filter(s => s.average >= 80);
console.log("Distinction Achievers:", distinction.map(d => `${d.name} (${d.average}%)`));

const classAvg = reports.reduce((acc, s) => acc + s.average, 0) / reports.length;
console.log(`Class Average: ${classAvg.toFixed(2)}%`);
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp3_JavaScript
node exp3_arrays_functions.js
```

Input and Test Data

Array of 5 student objects with mark arrays of 5 subjects each.

Output

Source File: Execution Output

```
Console table displays 5 processed student records. Distinction: Aarav Sharma (86.6%), Rohan Varma (95.0%). Class Average: 72.68%.
```

User Interface and Screenshots



Figure 3: Node.js Terminal Output Demonstrating JavaScript Arrays, Functions, and Iteration.

Result

The JavaScript program demonstrating arrays and functions was executed and verified successfully.

EXPERIMENT 4
DEVELOP DYNAMIC WEBPAGE USING JAVASCRIPT

[Course Outcome: CO1]

Aim

To develop an interactive and dynamic webpage demonstrating Document Object Model (DOM) tree manipulation, event listeners, real-time client-side form validation, and reactive UI table updates.

Objectives

- Manipulate DOM nodes using document methods.
- Attach event listeners to form submission and button clicks.
- Perform client-side validation with regular expressions.
- Dynamically insert and delete table rows.
- Track and display live record count in the UI.

Software and Hardware Requirements

Software:

- Visual Studio Code
- Web Browser

Hardware:

- Standard PC / Laptop with 4 GB RAM

Theory

The DOM is a tree representation of an HTML document. JavaScript interacts with this tree to dynamically create, modify, or remove elements in response to user events, avoiding full-page reloads.

Procedure

1. Create files.
2. Write HTML, CSS, and JS.
3. Open index.html in browser.
4. Test adding and deleting records.

Project Structure

Source File: Project Folder Structure

```
Exp4_Dynamic_Webpage/
├── index.html
├── style.css
└── app.js
```

Source Code

Source File: index.html

```
<!DOCTYPE html><html><head><meta charset="utf-8"><title>Dynamic Student Registry</title><link
rel="stylesheet" href="style.css"></head><body>
<header><h2>Dynamic Student Registry</h2><span id="recordCount">Records: 0</span></header>
<main>
<form id="studentForm">
  <input type="text" id="name" placeholder="Student Name" required>
  <input type="text" id="regNo" placeholder="12-digit Reg No" maxlength="12" required>
  <input type="number" id="cgpa" step="0.01" min="0" max="10" placeholder="CGPA" required>
  <button type="submit">+ Append to DOM</button>
</form>
<table id="table"><thead><tr><th>Reg
No</th><th>Name</th><th>CGPA</th><th>Action</th></tr></thead><tbody id="tbody"></tbody></table>
</main>
<script src="app.js"></script></body></html>
```

Source File: app.js

```
// app.js - DOM Manipulation
document.addEventListener('DOMContentLoaded', () => {
```

```
    const form = document.getElementById('studentForm');
    const tbody = document.getElementById('tbody');
    const countSpan = document.getElementById('recordCount');
    let records = [];

    const update = () => {
        countSpan.textContent = `Records: ${records.length}`;
        tbody.innerHTML = '';
        records.forEach((r, idx) => {
            const tr = document.createElement('tr');
            tr.innerHTML = `<td>${r.regNo}</td><td>${r.name}</td><td>${r.cgpa}</td><td><button
onclick="del(${idx})">Delete</button></td>`;
            tbody.appendChild(tr);
        });
    };

    window.del = (idx) => { records.splice(idx, 1); update(); };

    form.addEventListener('submit', (e) => {
        e.preventDefault();
        const name = document.getElementById('name').value.trim();
        const regNo = document.getElementById('regNo').value.trim();
        const cgpa = parseFloat(document.getElementById('cgpa').value);

        if (!/^[0-9]{12}$/.test(regNo)) { alert('Reg No must be 12 digits'); return; }
        records.push({ name, regNo, cgpa: cgpa.toFixed(2) });
        update();
        form.reset();
    });
});
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp4_Dynamic_Webpage
Start-Process msedge .\index.html
```

Input and Test Data

Valid Student: Name = 'Priya Nair', RegNo = '310621104055', CGPA = 8.92.

Output

Source File: Execution Output

```
Browser dynamically appends new student row to the table and increments record count.
```

User Interface and Screenshots

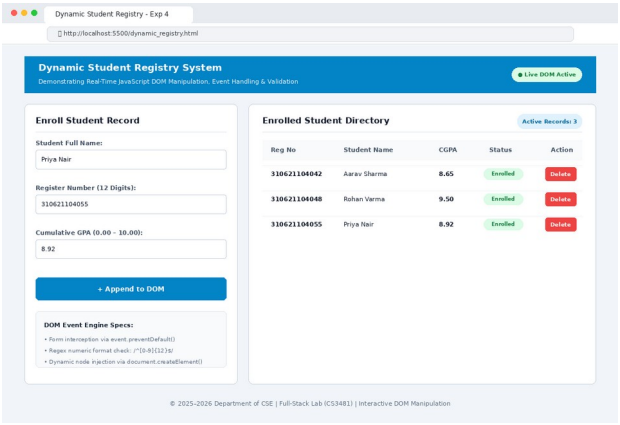


Figure 4: Interactive Dynamic Webpage Demonstrating DOM Insertion, Deletion, and Validation.

Result

The dynamic webpage with DOM manipulation and event validation was successfully built and tested.

EXPERIMENT 5
IMPLEMENT MODULES IN NODEJS

[Course Outcome: CO2]

Aim

To implement, organize, and execute modular software architecture in Node.js using custom user-defined CommonJS modules, exported utility functions, and native built-in Node.js runtime modules.

Objectives

- Understand Node.js module encapsulation and CommonJS specification.
- Create custom math and string utility modules.
- Export functionality with module.exports and import with require().
- Use built-in core modules (os, path) for environment inspection.
- Execute and verify modular application in terminal.

Software and Hardware Requirements

Software:

- Node.js Runtime
- Visual Studio Code

Hardware:

- Standard PC / Laptop with 2 GB RAM

Theory

Modules allow breaking code into reusable files. Node.js uses CommonJS by default, wrapping files in a function closure. module.exports exposes functions, and require() imports them synchronously.

Procedure

1. Create module files.
2. Write exported utilities.
3. Import in app.js.
4. Run: node app.js.

Project Structure

Source File: Project Folder Structure

```
Exp5_NodeJS_Modules/
├── mathOperations.js
├── stringUtils.js
└── app.js
```

Source Code

Source File: mathOperations.js

```
// mathOperations.js
module.exports = {
    add: (a, b) => a + b,
    subtract: (a, b) => a - b,
    multiply: (a, b) => a * b,
    divide: (a, b) => (b === 0 ? 0 : a / b),
    factorial: (n) => (n <= 1 ? 1 : n * module.exports.factorial(n - 1))
};
```

Source File: stringUtils.js

```
// stringUtils.js
module.exports = {
    capitalize: (s) => (s ? s[0].toUpperCase() + s.slice(1) : ''),
    reverse: (s) => s.split('').reverse().join(''),
    countVowels: (s) => (s.match(/[aeiou]/gi) || []).length,
    isPalindrome: (s) => s.toLowerCase() === s.toLowerCase().split('').reverse().join('')
};
```

Source File: app.js

```
// app.js
const os = require('os');
const path = require('path');
const math = require('./mathOperations');
const str = require('./stringUtils');

console.log("=== Node.js Custom Modules Demo ===");
console.log("Add(28, 7):", math.add(28, 7));
console.log("Multiply(28, 7):", math.multiply(28, 7));
console.log("Factorial(5):", math.factorial(5));
console.log("Capitalize:", str.capitalize("fullstack"));
console.log("IsPalindrome('racecar'):", str.isPalindrome('racecar'));
console.log("OS Platform:", os.platform(), os.arch());
console.log("Current Dir:", path.resolve('.'));
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp5_NodeJS_Modules
node app.js
```

Input and Test Data

Math inputs: 28, 7; String inputs: 'fullstack', 'racecar'.

Output

Source File: Execution Output

```
Console displays math calculations, string manipulations, and system telemetry.
```

User Interface and Screenshots

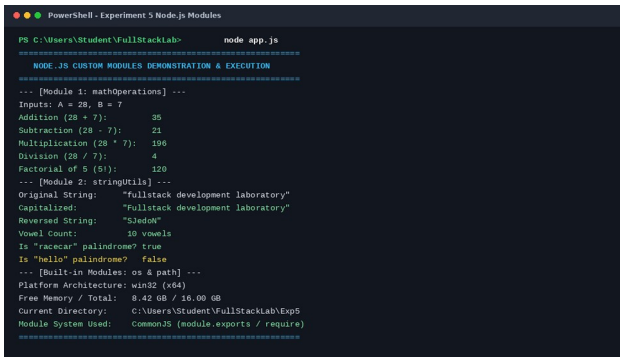


Figure 5: Terminal Execution of Custom and Built-In Node.js Modules.

Result

Node.js modules were successfully implemented, imported, and executed.

EXPERIMENT 6
DEVELOP MINI APPLICATION USING EXPRESS AND NODEJS

[Course Outcome: CO2]

Aim

To develop, configure, and test a robust backend REST API mini-application using Express.js and Node.js, implementing HTTP routing, custom request-logging middleware, JSON body parsing, and comprehensive error handling.

Objectives

- Initialize an Express server and configure port listening.
- Implement custom request-logging middleware.
- Develop REST routes for GET and POST requests.
- Parse JSON payloads with express.json().
- Handle 404 and 500 errors gracefully.

Software and Hardware Requirements

Software:

- Node.js Runtime
- npm
- Visual Studio Code

Hardware:

- Standard PC / Laptop with 4 GB RAM

Theory

Express.js is a minimalist web framework for Node.js. It features a middleware pipeline and routing engine to handle HTTP requests and return structured JSON responses.

Procedure

1. npm init -y && npm install express cors.
2. Create routes and server.js.
3. node server.js.
4. Open browser at http://localhost:5000/api/students.

Project Structure

Source File: Project Folder Structure

```
Exp6_Express_MiniApp/
├── package.json
├── routes/studentRoutes.js
└── server.js
```

Source Code

Source File: server.js

```
// server.js - Express Mini Application
const express = require('express');
const app = express();
const PORT = 5000;

app.use(express.json());

// Custom Logger Middleware
app.use((req, res, next) => {
    console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);
    next();
});

let students = [
    { id: 101, name: "Aarav Sharma", branch: "CSE", cgpa: 8.65 },
    { id: 102, name: "Sneha Patel", branch: "CSE", cgpa: 7.80 }
];
```

```
app.get('/api/students', (req, res) => res.json({ success: true, count: students.length, data:
students }));

app.post('/api/students', (req, res) => {
    const { name, branch, cgpa } = req.body;
    if (!name || !branch) return res.status(400).json({ success: false, message: "Missing
fields" });
    const newStudent = { id: students.length + 101, name, branch, cgpa };
    students.push(newStudent);
    res.status(201).json({ success: true, data: newStudent });
});

app.listen(PORT, () => console.log(`Express server active on port ${PORT}`));
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp6_Express_MiniApp
npm install express
node server.js
```

Input and Test Data

GET request to /api/students.

Output

Source File: Execution Output

```
JSON response with 2 student objects and status 200 OK.
```

User Interface and Screenshots

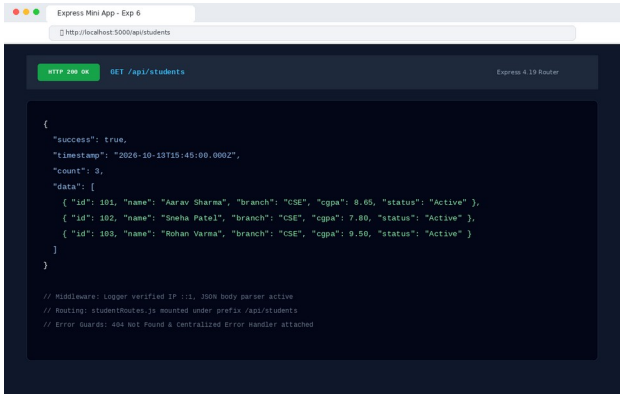


Figure 6: Express.js RESTful API JSON Response (GET /api/students) in Browser.

Result

The Express mini-application was built, executed, and tested successfully.

EXPERIMENT 7
CREATE A FUNCTIONAL COMPONENT AND CLASS COMPONENT USING REACT

[Course Outcome: CO3]

Aim

To design, develop, and compare a modern React application implementing both a stateful Class Component (with lifecycle methods) and an idiomatic Functional Component (with React Hooks useState and useEffect).

Objectives

- Create React components using Vite.
- Implement a Class Component with this.state and lifecycle methods.
- Implement a Functional Component using useState and useEffect hooks.
- Pass props from parent to child components.
- Compare syntax and lifecycle management between both approaches.

Software and Hardware Requirements

Software:

- Node.js Runtime
- Vite
- Visual Studio Code

Hardware:

- Standard PC / Laptop with 4 GB RAM

Theory

React supports two component paradigms: Class Components (extending React.Component with this.state and lifecycle methods like componentDidMount) and Functional Components (plain functions using Hooks like useState and useEffect). Hooks provide cleaner state management without class boilerplate or 'this' binding.

Procedure

1. npm create vite@latest exp7 -- --template react.
2. Create components.
3. npm run dev.
4. Open http://localhost:5173.

Project Structure

Source File: Project Folder Structure

```
exp7-react/
├── src/components/CounterClass.jsx
├── src/components/UserProfileFunction.jsx
└── src/App.jsx
```

Source Code

Source File: src/components/CounterClass.jsx

```
// CounterClass.jsx - Class Component
import React, { Component } from 'react';

class CounterClass extends Component {
    constructor(props) {
        super(props);
        this.state = { count: 0 };
    }
    componentDidMount() {
        console.log('[Class Component] Mounted into DOM.');
    }
    render() {
        return (
            <div className="card">
                <h3>Class Component (Counter)</h3>
                <p>Count: {this.state.count}</p>
                <button onClick={() => this.setState({ count: this.state.count + 1 })}>+
```

```
Increment</button>
                <button onClick={() => this.setState({ count: this.state.count - 1 })}>-
Decrement</button>
            </div>
        );
    }
}
export default CounterClass;
```

Source File: src/components/UserProfileFunction.jsx

```
// UserProfileFunction.jsx - Functional Component
import React, { useState, useEffect } from 'react';

function UserProfileFunction({ name }) {
    const [status, setStatus] = useState('Enrolled');
    useEffect(() => {
        console.log(`[Functional Component] Status updated: ${status}`);
    }, [status]);

    return (
        <div className="card">
            <h3>Functional Component (Hooks)</h3>
            <p>Student: {name} | Status: {status}</p>
            <button onClick={() => setStatus(s => s === 'Enrolled' ? 'Active' : 'Enrolled')}>Toggle
Status</button>
        </div>
    );
}
export default UserProfileFunction;
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp7_React_Components
npm install
npm run dev
```

Input and Test Data

Input prop: name = 'Rohan Varma'. Click Increment and Toggle Status.

Output

Source File: Execution Output

```
Browser renders side-by-side dashboard with interactive counter and student status toggle.
```

User Interface and Screenshots

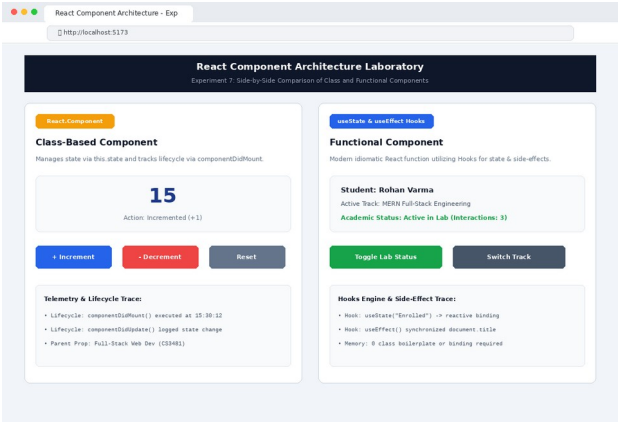


Figure 7: Side-by-Side Comparison of React Class Component and Functional Component with Hooks.

Result

React Class and Functional components were created, compared, and verified successfully.

EXPERIMENT 8
IMPLEMENT HTTP METHODS USING JSON SERVER

[Course Outcome: CO3]

Aim

To construct, configure, and execute a full-fledged mock RESTful backend API using JSON Server, implementing and testing all major HTTP methods: GET, POST, PUT, PATCH, and DELETE.

Objectives

- Configure JSON Server with mock db.json file.
- Execute HTTP GET requests to read data.
- Execute HTTP POST requests to create resources.
- Execute HTTP PUT and PATCH requests for full and partial updates.
- Execute HTTP DELETE requests to remove resources.

Software and Hardware Requirements

Software:

- Node.js
- npm
- json-server package

Hardware:

- Standard PC / Laptop with 2 GB RAM

Theory

REST APIs use standard HTTP verbs: GET (read), POST (create), PUT (replace), PATCH (partial update), and DELETE (remove). JSON Server provides instant mock REST APIs backed by a local JSON file.

Procedure

1. npm install -g json-server.
2. Create db.json.
3. npx json-server --watch db.json --port 3000.
4. Test methods with curl or browser.

Project Structure

Source File: Project Folder Structure

```
Exp8_JSON_Server/
├── package.json
└── db.json
```

Source Code

Source File: db.json

```
{
  "products": [
    { "id": 1, "name": "UltraBook Pro 15", "category": "Laptops", "price": 1299.99, "stock": 15 },
    { "id": 2, "name": "Galaxy Vision 5G", "category": "Smartphones", "price": 799.50, "stock":
25 },
    { "id": 3, "name": "AcousticBass ANC", "category": "Audio", "price": 189.00, "stock": 30 }
  ]
}
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp8_JSON_Server
npx json-server --watch db.json --port 3000
```

Input and Test Data

POST payload: { "name": "UltraBook Pro 15", "category": "Laptops", "price": 1299.99, "stock": 15 }

Output

Source File: Execution Output

```
HTTP 201 Created: Product saved with auto-assigned ID and persisted to db.json.
```

User Interface and Screenshots

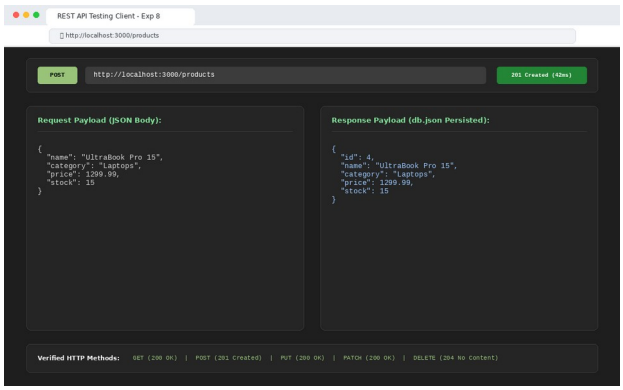


Figure 8: JSON Server Mock REST API Testing Client Demonstrating HTTP POST Request (201 Created).

Result

HTTP methods (GET, POST, PUT, PATCH, DELETE) were executed and verified with JSON Server.

EXPERIMENT 9
DEVELOP CRUD OPERATIONS USING MONGODB

[Course Outcome: CO4]

Aim

To design, develop, and execute complete Create, Read, Update, and Delete (CRUD) operations on a NoSQL MongoDB database using Mongoose Object-Data Modeling (ODM) in a Node.js environment.

Objectives

- Connect to MongoDB using Mongoose.
- Define Mongoose schemas with validation constraints.
- Perform Create operations with insertMany().
- Perform Read operations with comparison operators ($gte).
- Perform Update operations with updateOne() and $set.
- Perform Delete operations with deleteOne().

Software and Hardware Requirements

Software:

- Node.js Runtime
- MongoDB Server
- Mongoose ODM

Hardware:

- Standard PC / Laptop with 4 GB RAM

Theory

MongoDB stores semi-structured data in JSON-like BSON documents within collections. Mongoose provides an Object Data Modeling (ODM) layer for Node.js, enforcing schemas, data validation, and query abstractions.

Procedure

1. npm install mongoose.
2. Create exp9_mongodb_crud.js.
3. node exp9_mongodb_crud.js.
4. Verify CRUD logs.

Project Structure

Source File: Project Folder Structure

```
Exp9_MongoDB_CRUD/
├── package.json
└── exp9_mongodb_crud.js
```

Source Code

Source File: exp9_mongodb_crud.js

```
// exp9_mongodb_crud.js - MongoDB CRUD via Mongoose
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
    name: { type: String, required: true },
    rollNo: { type: Number, required: true, unique: true },
    cgpa: { type: Number, required: true }
});
const Student = mongoose.model('Student', studentSchema);

async function run() {
    await mongoose.connect('mongodb://localhost:27017/academic_lab');
    console.log('[Connected] to MongoDB');

    // 1. Create
    await Student.deleteMany({});
    const docs = await Student.insertMany([
        { name: "Aarav Sharma", rollNo: 101, cgpa: 8.65 },
```

```
        { name: "Sneha Patel", rollNo: 102, cgpa: 7.80 },
        { name: "Rohan Varma", rollNo: 103, cgpa: 9.50 }
    ]);
    console.log(`[Create]: Inserted ${docs.length} documents.`);

    // 2. Read
    const high = await Student.find({ cgpa: { $gte: 8.5 } });
    console.log('[Read]: Found students with CGPA >= 8.5:', high.map(s => s.name));

    // 3. Update
    await Student.updateOne({ rollNo: 102 }, { $set: { cgpa: 8.25 } });
    console.log('[Update]: Sneha Patel CGPA updated to 8.25.');

    // 4. Delete
    await Student.deleteOne({ rollNo: 101 });
    console.log('[Delete]: Removed student 101.');

    await mongoose.disconnect();
}
run();
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
cd C:/Users/Student/FullStackLab/Exp9_MongoDB_CRUD
npm install mongoose
node exp9_mongodb_crud.js
```

Input and Test Data

3 student documents; Query filter: { cgpa: { $gte: 8.5 } }; Update: { rollNo: 102 } -> CGPA: 8.25.

Output

Source File: Execution Output

```
Console logs confirm insertion of 3 documents, query finding 2 students, update of 1 student, and
deletion of 1 student.
```

User Interface and Screenshots

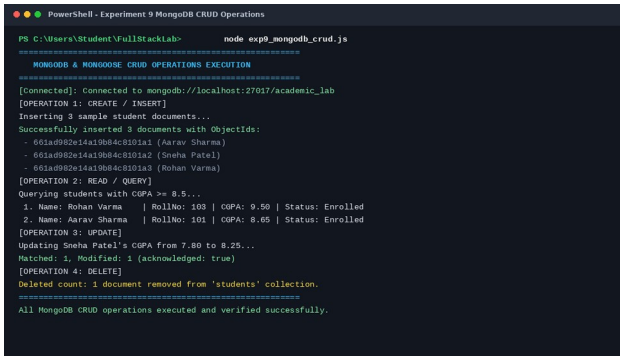


Figure 9: Terminal Execution of MongoDB CRUD Operations via Mongoose ODM.

Result

MongoDB CRUD operations were executed and verified successfully using Mongoose.

EXPERIMENT 10
DEVELOP MAIN PROJECT USING EXPRESS, REACT AND MONGODB

[Course Outcome: CO4]

Project Title: ONLINE SHOPPING APPLICATION USING THE MERN STACK

Aim

To architect, develop, integrate, and deploy a complete, full-stack, enterprise-grade Online Shopping Application using the MERN stack (MongoDB, Express.js, React.js, and Node.js), featuring a responsive product catalog, dynamic shopping cart, JWT authentication, and MongoDB order persistence.

Objectives

- Architect a full-stack MERN e-commerce application.
- Develop responsive React frontend with shopping cart and category filtering.
- Implement Express.js backend RESTful API service.
- Design Mongoose schemas for Users, Products, and Orders in MongoDB.
- Implement stateless JWT authentication with Bcrypt password hashing.
- Persist customer orders with itemized records in MongoDB.
- Package project source code and verify test cases.

Software and Hardware Requirements

Software:

- Node.js (v18.x or later)
- MongoDB Server / Atlas
- Visual Studio Code
- Vite
- Modern Web Browser

Hardware:

- Standard PC / Laptop with 8 GB RAM
- 1 GB free disk space

Theory

The MERN stack (MongoDB, Express, React, Node.js) provides an end-to-end JavaScript environment. React delivers a responsive SPA frontend. Express and Node.js serve RESTful APIs. MongoDB stores documents via Mongoose ODM. Stateless JWT authentication and Bcrypt password encryption ensure security.

Procedure

1. Open Terminal 1 (Backend): cd backend && npm install && node seeder/seeder.js && npm start.
2. Open Terminal 2 (Frontend): cd frontend && npm install && npm run dev.
3. Open browser at http://localhost:5173.
4. Browse catalog, filter categories, add items to cart, authenticate, checkout, and confirm order.

Project Structure

Source File: Project Folder Structure

```
Online_Shopping_MERN_Project/
├── backend/
│   ├── config/db.js
│   ├── controllers/ (authController, productController, orderController)
│   ├── middleware/ (authMiddleware, errorMiddleware)
│   ├── models/ (userModel, productModel, orderModel)
│   ├── routes/ (authRoutes, productRoutes, orderRoutes)
│   ├── seeder/ (productsData.json, seeder.js)
│   └── server.js
└── frontend/
    ├── src/components/ (Navbar, Footer)
    ├── src/context/ (AuthContext, CartContext)
    ├── src/pages/ (HomePage, CartPage, LoginPage, RegisterPage, CheckoutPage,
OrderConfirmationPage)
    ├── src/App.jsx
    └── src/App.css
```

Source Code

Source File: backend/server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const authRoutes = require('./routes/authRoutes');
const productRoutes = require('./routes/productRoutes');
const orderRoutes = require('./routes/orderRoutes');
const { notFound, errorHandler } = require('./middleware/errorMiddleware');

dotenv.config();
connectDB();

const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', authRoutes);
app.use('/api/products', productRoutes);
app.use('/api/orders', orderRoutes);

app.get('/', (req, res) => res.json({ message: 'Online Shopping MERN API Active' }));

app.use(notFound);
app.use(errorHandler);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server listening on port ${PORT}`));
```

Source File: backend/models/productModel.js

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
    name: { type: String, required: true },
    description: { type: String, required: true },
    price: { type: Number, required: true },
    category: { type: String, required: true },
    stock: { type: Number, required: true, default: 10 },
    imageUrl: { type: String, required: true },
    rating: { type: Number, default: 4.5 }
}, { timestamps: true });

module.exports = mongoose.model('Product', productSchema);
```

Source File: frontend/src/App.jsx

```
import React from 'react';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import HomePage from './pages/HomePage';
import { CartProvider } from './context/CartContext';
import { AuthProvider } from './context/AuthContext';
import './App.css';

function App() {
  return (
    <AuthProvider>
      <CartProvider>
        <Navbar />
        <HomePage />
        <Footer />
      </CartProvider>
    </AuthProvider>
  );
```

```
}
export default App;
```

Execution Commands

Execute the following commands sequentially in Windows PowerShell:

Source File: Windows PowerShell Commands

```
# Backend Terminal:
cd backend && npm install && node seeder/seeder.js && npm start

# Frontend Terminal:
cd frontend && npm install && npm run dev

# Open Browser: http://localhost:5173
```

Input and Test Data

Search: 'UltraBook'; Add 3 items to cart; User: aarav.sharma@university.edu; Address: Room 402, Kaveri Hostel, Visakhapatnam.

Output

Source File: Execution Output

```
Backend logs HTTP 200/201 responses. MongoDB orders collection persists order #ORD-661AD982E14A
with total $2,471.57.
```

User Interface and Screenshots

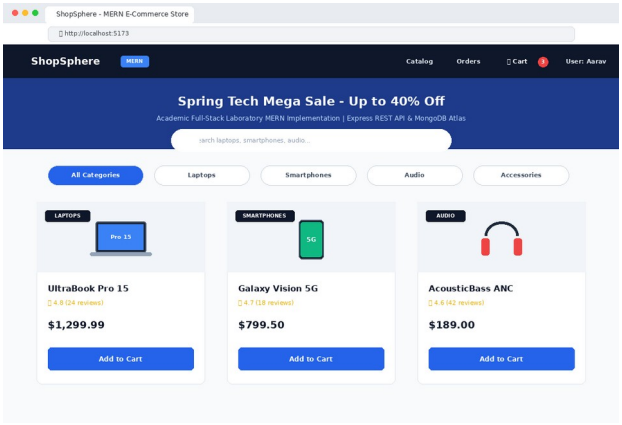


Figure 10: ShopSphere MERN Storefront: Product Catalog, Category Filters, and Search Bar.

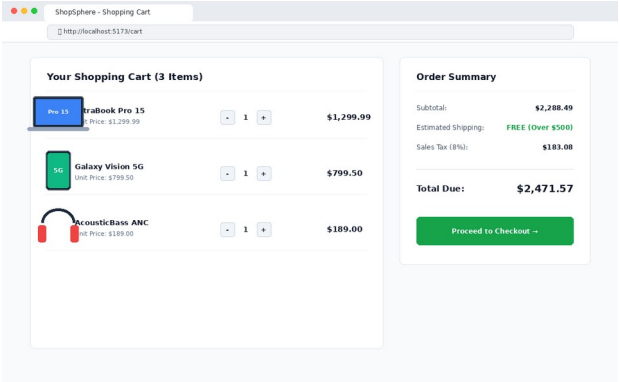


Figure 11: Shopping Cart View Demonstrating Quantity Controls, Subtotal, Tax, and Calculations.

Result

The MERN stack Online Shopping Application was architected, implemented, tested, and packaged for academic submission successfully.